
Milestone-2 Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Source Code

The complete implementation is hosted at: <https://github.com/chiron-verification/milestone-2>

- **Milestone-1 baseline commit:** 0f99ade6d4aa2516f9e47e013ad06d29e8b8a2f7
- **Milestone-2 submission commit:** <TBD>

Core source files and responsibilities:

- **safety_properties.py:** Main verification API; parses user properties, applies checks/hints, and reports PASSED/FAILED/UNKNOWN with timing, and now refactored from an interactive CLI to callable `CHC_Verification(...)` with structured `ReturnValue`, timeout handling, property-scope control, and heading-grid pre-check/hint orchestration.
- **step_rules.py:** Encodes IR instructions into CHC transition rules used by the fixedpoint engine, with replaced trig bucket bounds with exact 15-degree trig encoding, introduced `BadHeading` rule generation, and added optimization-aware loop summarization and turn-safety shortcuts.
- **init_fixed_point.py:** Initializes solver state and base relations for default, specific, and universal modes, and now registers `BadHeading`, propagates `optimization_level`, constrains universal initialization to heading-grid and integer user variables
- **z3_fixed_point.py:** Defines invariant/state-vector relations and mode-aware predicate structure, and now adds `optimization_level` plumbing and introduces the dedicated `BadHeading` relation in the same state signature as `Inv`.
- **variable_name_detection_in_IR.py:** Extracts variables from IR for building constraints and queries, and now introduces the shared `OptimizationLevel` enum and updates parser signatures/import path setup to align with the standalone CHC-Verification package layout.
- **heading_grid.py:** Encodes heading-grid checks to handle 15-degree alignment assumptions explicitly, and now added as a dedicated helper module exposing `heading_on_grid(h)` for reuse across initialization, transition encoding, and API-level assumptions.

- `optimization_helpers.py`: Provides static-analysis helpers for optimization decisions and loop simplification metadata, introduced to implement turn-safety dataflow, repeat-loop pattern detection, and summarizability checks used by `OptimizationLevel.BASIC`.

The correctness test suite (`correctness_tests/`) has substantially expanded across `test_default.py`, `test_specific.py`, and `test_universal.py`, along with many new `.tl` fixtures and shared helpers to improve coverage of arithmetic, control-flow, and heading-related properties.

In addition, a dedicated performance testing setup (`performance_tests/`) is added with mode-wise test runners, benchmark fixtures, and reusable harness/configuration files to measure solver behavior on deep nesting, wide-state, branch-heavy, and high-iteration workloads.

Current Progress

Overview of the progress made since the first milestone is as follows:

- **Pipeline refactoring into a callable API:** The verification flow is now exposed through `CHC_Verification(...)` with structured output fields (status, advice, timing, and per-property results) instead of an interactive-only CLI path.
- **Optimization pipeline added:** A `BASIC` optimization mode adds static IR analysis, turn-safety reasoning, and summarizable-loop compression before CHC querying, while `OptimizationLevel.NONE` retains the conservative baseline behavior.
- **Property scope feature added:** Properties can now be checked over all reachable states or restricted to terminating states only (`property_scope = "terminating"`).
- **Heading-grid safety checks integrated:** The verifier now models and queries heading-grid safety explicitly through a dedicated `BadHeading` relation before property checking.
- **Heading-grid hints integrated:** Users can either request verification of the heading-grid invariant (`check_heading_always_on_grid`) or force an assumption (`heading_on_grid_always`) for faster runs on hard cases.
- **Correctness tests:** *TODO*
- **Performance tests:** *TODO*

See Figure 1 for a visual overview of the main verification pipeline stages, responsibilities, and data flow, and the subsequent sections for detailed implementation notes on each major feature addition and refactor. The details of correctness and performance testing can be found in the Tests section.

Main Pipeline Flow (Responsibilities and Data Flow)

Implementation Details: Callable Verification API

File: `CHC-Verification/safety_properties.py`

The prior interactive `__main__` flow was replaced by a callable entry point:

```
CHC_Verification(
    file_name,
    mode,
    user_properties,
    params=None,
    property_scope="all",
    hints=["check_heading_always_on_grid"],
    timeout_ms=60_000,
    optimization_level=OptimizationLevel.NONE,
)
```

This API returns a `ReturnValue` object carrying: `status`, `heading_grid_safe`, `build_time`, `solve_times`, `passing_properties`, `failing_properties`, and `unknown_properties`. The implementation first validates mode, scope, hint format, and specific-mode parameters, then builds the fixedpoint, applies assumptions/pre-checks, and evaluates properties in a controlled `eval` context with restricted builtins.

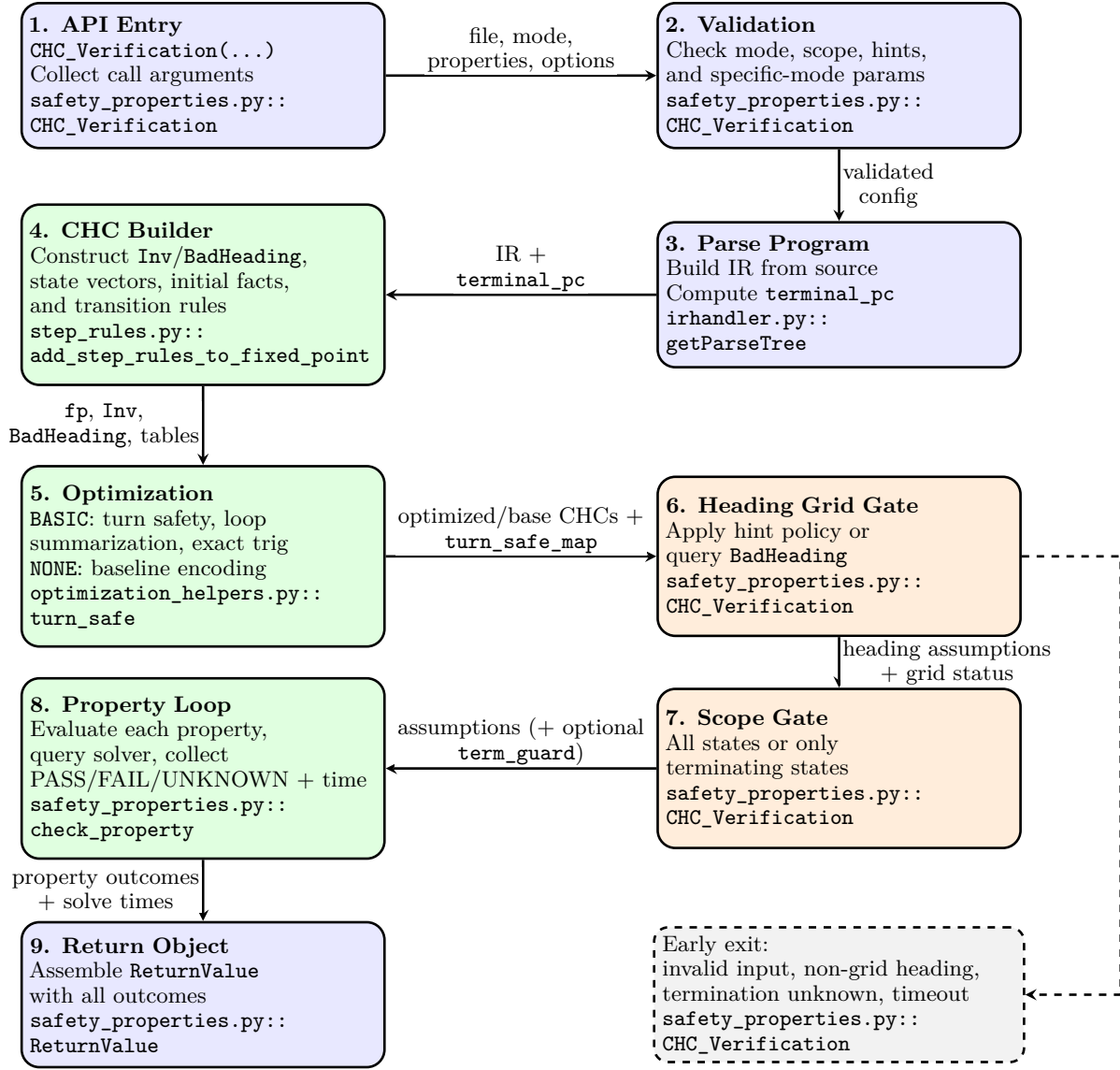


Figure 1: Main verification pipeline showing stage responsibilities and data artifacts flowing through parsing, CHC construction, heading/scope controls, and final result aggregation.

Implementation Details: Optimization Pipeline

Files: CHC-Verification/optimization_helpers.py, CHC-Verification/step_rules.py

New optimization layer with two supported modes. In `OptimizationLevel.BASIC`, the verifier computes a static environment over IR control-flow to detect turn operations that provably preserve the 15-degree grid (`turn_safe`). It also detects canonical repeat-loop shapes through `find_repeat_loops(...)` and admits only structurally safe loops via `is_summarizable_loop(...)`.

For those loops, `step_rules.py` synthesizes a summarized transition from loop-entry PC directly to loop-exit PC and emits any required summarized `BadHeading` obligations. For non-summarized code, the original instruction-wise rule generation remains active. This preserves soundness while reducing rule count and symbolic clutter on common deterministic repeat patterns.

An additional semantic tightening is that movement trig handling was upgraded from interval over-approximation buckets to exact 15-degree lookup values (`cos_sin_exact_z3`), aligning the transition encoding

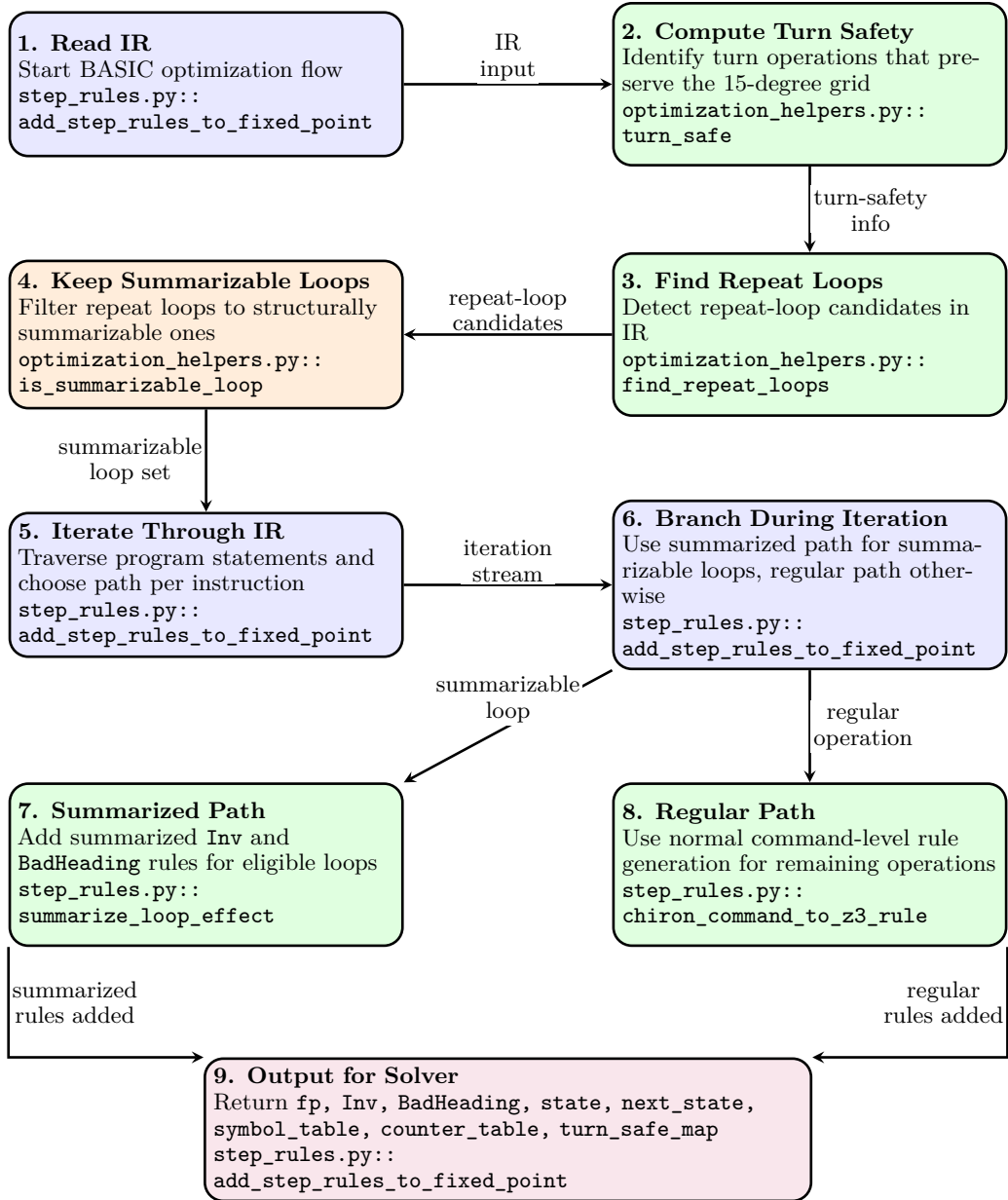


Figure 2: BASIC optimization workflow: read IR, compute turn safety, detect and filter repeat loops to summarizable loops, then during iteration emit summarized Inv/BadHeading rules for eligible loops and regular rules for other operations.

with the new heading-grid reasoning. The `OptimizationLevel.NONE` mode bypassed the optimization pipeline and performs standard verification. See Figure 2 for a visual overview of the optimization flow.

Implementation Details: Property Scope

File: `CHC-Verification/safety_properties.py`

The new `property_scope` input supports "all" and "terminating". For terminating-only checks, the implementation:

1. computes terminal PC as `len(ir)`;
2. checks reachability of terminating states using `Exists(vars, Inv(state) & term_guard)`;

3. returns UNKNOWN early if termination is unreachable/unknown;
4. conjoins `term_guard` into assumptions for all property queries.

This keeps a single property-checking loop while changing the semantic domain of verification in a principled way.

Implementation Details: Heading-grid Safety Check

Files:

1. *CHC-Verification/z3_fixed_point.py*
2. *CHC-Verification/init_fixed_point.py*
3. *CHC-Verification/step_rules.py*
4. *CHC-Verification/heading_grid.py*

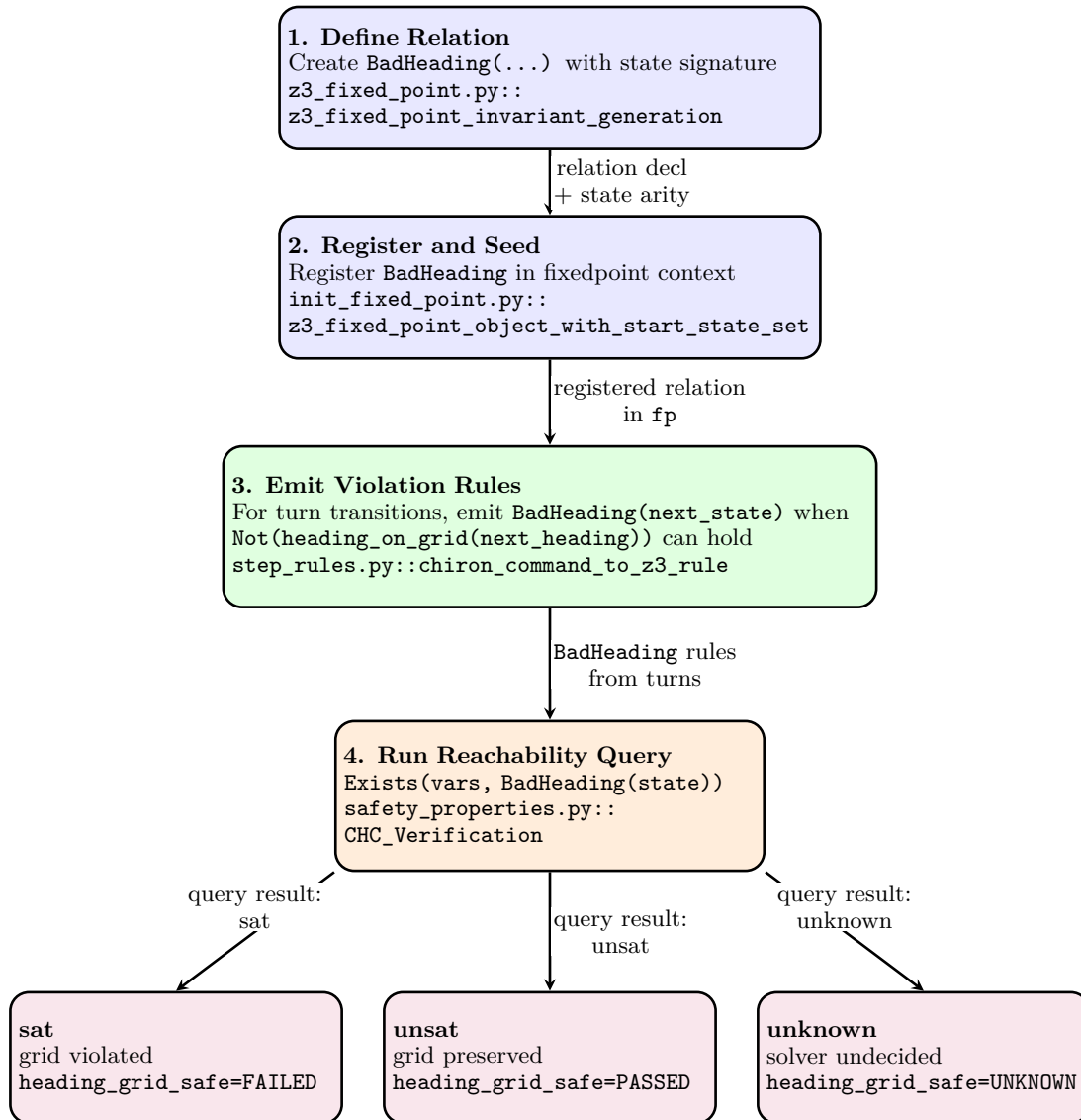


Figure 3: BadHeading workflow for proving whether heading remains on the 15-degree grid and mapping query outcomes to verification status.

The new pipeline introduces a dedicated relation `BadHeading(...)` at the invariant signature level and registers it in the fixedpoint context. Transition rules for turn commands can emit `BadHeading` consequences whenever `Not(heading_on_grid(next_heading))` is reachable, see Figure 3 for the workflow.

At API level, when checking is requested, the verifier queries `Exists(vars, BadHeading(state))`. The result semantics are: `sat -> heading_grid_safe = FAILED (overall UNKNOWN)`, `unsat -> heading_grid_safe = PASSED (continue)`, `unknown -> heading_grid_safe = UNKNOWN (overall UNKNOWN)`.

This turns an implicit modeling assumption into an explicit proof obligation, improving interpretability of trig-sensitive verification outcomes.

Implementation Details: Heading-grid Hints

File: CHC-Verification/safety_properties.py

Hints are parsed as a `StrEnum` with values `check_heading_always_on_grid` and `heading_on_grid_always`. The implementation validates hint type and value, then applies a precedence rule:

- if `heading_on_grid_always` is present, the solver skips the heading-grid query and directly injects `heading_on_grid(state[3])` into assumptions;
- otherwise, if `check_heading_always_on_grid` is present, the explicit `BadHeading` query is performed.

In `OptimizationLevel.BASIC`, if static analysis proves every turn is safe (`is_all_turn_safe`), the heading-grid check short-circuits to `PASSED`, avoiding unnecessary solver effort.

API Usage Guide

The expected calling pattern is:

```
from safety_properties import CHC_Verification
from variable_name_detection_in_IR import OptimizationLevel

class UserProperty:
    def __init__(self, name, expr):
        self.name = name
        self.expr = expr

props = [UserProperty("x_nonneg", "x >= 0")]

result = CHC_Verification(
    "correctness_tests/programs/loop_accum.tl",
    "default",
    props,
    property_scope="all",
    hints=["check_heading_always_on_grid"],
    timeout_ms=60_000,
    optimization_level=OptimizationLevel.BASIC,
)

print("Result:", result.status)
print("Heading Grid Safe:", result.heading_grid_safe)
print("Build Time:", result.build_time)
print("Solve Times:", result.solve_times)
```

Operational notes:

- Use `specific` mode with a parameter dictionary string (e.g., `"'x': 10"`);
- Use `property_scope="terminating"` only when termination-focused properties are intended;
- Prefer `check_heading_always_on_grid` unless the program is known to be heading-grid safe or the check is not relevant, as provides a good balance of soundness and performance.

Tests

Correctness Tests

The primary evaluation artifacts for this milestone are the automated test modules under `CHC-Verification/correctness_tests/`, the shared helper framework, and the `.tl` fixture-program corpus.

Test files used for evaluation

File	Role in evaluation
tests/test_default.py	Validates safety checking from the concrete default start state. Covers arithmetic, geometry, pen-state, directional, heading-grid, trig-dependent, and advanced nested-loop programs.
tests/test_specific.py	Validates parameterized initialization semantics. Focuses on boundary-sensitive properties where outcomes depend on user-supplied input dictionaries.
tests/test_universal.py	Validates unconstrained-initial-state reasoning, API error handling, terminating-only checks, and relational properties over symbolic inputs.
tests/helpers.py	Shared harness providing fixedpoint construction, context generation, API wrappers, property assertions, and output suppression for deterministic test behavior.
tests/README.md	Human-readable documentation of suite layout, categories, and execution workflow.

Coverage snapshot

- Total automated test cases: **202**
- Fixture programs in `tests/programs/`: **59** .tl files
- Explicitly skipped tests: **4** (timeout-marked trig/complex-heading cases)

Representative fixture-program families

How to run

Performance Tests

Analysis

Data and Visual Analysis

Strengths of the implementation

Limitations of the implementation

Future Plans

Milestone summary

Planned work

Declaration

We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course. We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment. We understand that any violation of this declaration may result in disciplinary action.